



AMD DEVELOPER HACKATHON • TRACK 1

TokenGolf

Track 1 ranks by **fewest tokens**. The catch: the only allowed models are **reasoning models** that burn hundreds of hidden tokens per answer. We turned that off. **-46% scored tokens, higher accuracy.**

\$ Fireworks AI • llama.cpp • Docker linux/amd64

fewest strokes wins

\$ THE GAME

Clear an accuracy gate, then rank by **fewest Fireworks tokens**. The only lever that helps is spending less.

THE TRAP

Both allowed models, minimax-m3 and kimi-k2p7-code, are **reasoning models**. They think out loud in hidden tokens before answering, and every token is scored.

THE COST

A one-word sentiment label cost **41–68 output tokens** of invisible reasoning. Across eight categories, that hidden thinking was our single biggest scored cost.

Everyone pays the accuracy gate. **The rank is won on tokens nobody sees.**

\$ THE STROKE

One flag flips it: **reasoning_effort = "none".**

sentiment "positive" 41-68 tok → 2 tok

kiml mean / answer 341 tok → 185 tok (-46%)

accuracy .917 → .958 (went up)

Measured, not assumed. "low" was ignored by these models; chat_template_kwargs{enable_thinking:false} returned 400.
"none" is the only lever that fires, and it costs no accuracy.

Read the task, take **one** shot, write the answer.

01 Read `/input/tasks.json`

Eight categories: factual QA, math, sentiment, summarisation, NER, code-debug, logic, code-gen.

02 Route: free local first, else one Fireworks call

Optional local tier (Qwen2.5-3B on CPU, **0 scored tokens**) answers sentiment & NER. Everything else takes a single Fireworks call with reasoning off.

03 Binary cascade, never a chain

Because every call adds tokens, we never escalate model-to-model. Local → one remote call, terse and capped. That's it.

04 Write `/output/results.json`

Exact harness contract: every call routed via `FIREWORKS_BASE_URL`, models from `ALLOWED_MODELS`.

\$ KNOWING WHEN TO ESCALATE

A calibrated gate spends a paid call **only when it isn't sure.**

FREE CONFIDENCE

The local model answers a few times; agreement across samples is a self-consistency confidence signal, and sampling on local costs nothing.

CALIBRATED, NOT GUESSED

That signal is calibrated on held-out data, so the escalation threshold is measured rather than hand-tuned. High confidence → keep the free answer; low → escalate.

The honest default when unsure is to **spend one token-cheap Fireworks call**, not bluff a free answer that fails the gate.

\$ THE SCOREBOARD

Fewer tokens, **and** a higher gate score.

-46%

mean scored tokens
(kimi, 341 → 185)

.958

accuracy with reasoning off
(up from .917)

259

tokens on a live 3-task run
(was 493, same answers)

8/8

categories handled
through one contract

`Ships as linux/amd64 · cold-start 1.7s (limit 60s) · slowest request 4.4s (limit 30s) · image < 10 GB · 199 tests green.`

\$ BEYOND THE LEADERBOARD

Token efficiency is just **cost**, wearing a leaderboard's clothes.

- **Same answer, ~46% cheaper.** On any token-metered API, fewer tokens means a proportionally smaller inference bill for identical output quality.
- **A silent tax, everywhere.** Any team running reasoning models per token pays for hidden reasoning they never see; turning it off where accuracy holds is free money.
- **The pattern travels.** A calibrated local→remote cascade with an honest abstain is a reusable template for cost-optimal serving, not a one-off hack.

\$ STATED HONESTLY

What we measured, and what we **didn't overclaim**.

- ✓ Accuracy numbers are **n=3 per category**. The token cut is robust and per-call verified; the accuracy edge is directional, not proven.
- ✓ Latency was **de-risked by proxy** on our machine, not the real judging VM. Comfortable even at 5× slower, but stated as a proxy.
- ✓ Grammar-constrained decoding (from a paper sweep) was **tested and reverted**: 0 fixed, 1 broken on our checkers. A measured negative, kept out.
- ✓ The free-local tier's edge **shrank** once reasoning was off, so we lead with the lean Fireworks image and treat local as optional.

Reason less, score more.

TokenGolf: a calibrated, token-efficient routing agent for AMD Track 1.

image	<code>docker.io/soren19/router-agent: kimi</code>
code	<code>github.com/nguiaSoren/router-agent</code>